

Doc 10.1 — Function Fundamentals & Return Values

JavaScript-এর সবচেয়ে important building blocks-এর একটি হলো **Function** ।

Function বুঝে গেলে পরে **Arrow Functions, Callbacks, Higher-Order Functions, Closures, Promises, Async/Await, React event handlers**—সব অনেক সহজ হয়ে যাবে ।

1. Function কী?

Function হলো reusable code block, যেটাকে প্রয়োজন অনুযায়ী বারবার execute করা যায় ।

সহজভাবে:

একটা নির্দিষ্ট কাজ করার জন্য **code**-কে একটা নাম দিয়ে রাখা = **Function**

Example:

```
function greet() {  
  console.log("Hello!");  
}
```

এখন function-টা call করলে:

```
greet();
```

Output:

```
Hello!
```

আবার call করতে পারি:

```
greet();  
greet();  
greet();
```

Output:

```
Hello!  
Hello!  
Hello!
```

একবার function লিখে অনেকবার ব্যবহার করা যায়।

2. কেন Function ব্যবহার করি?

Function-এর প্রধান সুবিধা:

- Code reuse
- Code organization
- Duplicate code কমানো
- Complex problem ছোট অংশে ভাগ করা
- Maintainability
- Testing সহজ করা
- Readability বাড়ানো
- একই logic বিভিন্ন জায়গায় ব্যবহার করা

ধরো:

```
console.log("Welcome Shariar");  
console.log("Welcome Rahim");  
console.log("Welcome Karim");
```

এখানে repetitive code আছে।

Function ব্যবহার করলে:

```
function welcome(name) {  
  console.log(`Welcome ${name}`);  
}  
  
welcome("Shariar");  
welcome("Rahim");  
welcome("Karim");
```

এখন logic এক জায়গায় আছে।

3. Function Declaration

সবচেয়ে basic function syntax:

```
function greet() {  
  console.log("Hello");  
}
```

এটাকে বলা হয়:

Function Declaration

Structure:

```
function  
  ↓  
function name  
  ↓  
parameters  
  ↓  
function body
```

Example:

```
function add(a, b) {  
  return a + b;  
}
```

এখানে:

```
function → keyword  
add      → function name  
a, b     → parameters  
return   → returned result
```

4. Function Call / Invocation

Function define করলেই function execute হয় না।

```
function greet() {  
  console.log("Hello");  
}
```

শুধু definition তৈরি হয়েছে।

Execute করতে হবে:

```
greet();
```

এটাকে বলা হয়:

- Function call
- Function invocation

দুটো term-ই দেখতে পাবে।

5. Function Definition vs Function Call

খুব important distinction:

Definition

```
function greet() {  
  console.log("Hello");  
}
```

মানে:

Function তৈরি করছি।

Call

```
greet();
```

মানে:

Function execute করছি।

Mental model:

Define → Function তৈরি
Call → Function execute

6. Function Parameters

Function-এর ভিতরে external data পাঠানোর জন্য **parameters** ব্যবহার করি।

```
function greet(name) {  
  console.log(`Hello ${name}`);  
}
```

এখানে:

name

হলো parameter।

Call:

```
greet("Shariar");
```

Output:

Hello Shariar

7. Parameters vs Arguments

এটা খুব important terminology।

```
function greet(name) {  
  console.log(name);  
}  
  
greet("Shariar");
```

এখানে:

name

= Parameter

আর:

"Shariar"

= Argument

সহজভাবে

Parameter
→ Function definition-এর variable

Argument
→ Function call-এর সময় পাঠানো actual value

আরেকটা example:

```
function add(a, b) {  
  return a + b;  
}  
  
add(10, 20);
```

এখানে:

a, b
→ parameters

10, 20
→ arguments

8. Multiple Parameters

একাধিক parameter থাকতে পারে।

```
function calculateTotal(price, quantity) {  
  return price * quantity;  
}  
  
const total = calculateTotal(500, 3);  
  
console.log(total);
```

Output:

1500

এখানে:

```
price    → parameter  
quantity → parameter  
  
500      → argument  
3        → argument
```

9. Function Expression

Function declaration ছাড়াও function তৈরি করা যায়।

```
const greet = function () {  
  console.log("Hello");  
};
```

এটাকে বলা হয়:

Function Expression

এখানে function-টাকে একটি variable-এর মধ্যে assign করা হয়েছে।

10. Function Declaration vs Function Expression

Function Declaration

```
function greet() {  
  console.log("Hello");  
}
```

Function Expression

```
const greet = function () {  
  console.log("Hello");  
};
```

দুটাই function তৈরি করে।

কিন্তু syntax এবং hoisting behavior-এর মধ্যে difference আছে।

11. Function Declaration-এর Hoisting

Function declaration hoist করা হয়।

তাই declaration-এর আগে call করা যায়:

```
greet();  
  
function greet() {  
  console.log("Hello");  
}
```

Output:

```
Hello
```

JavaScript function declaration-কে execution-এর আগে available করে দেয়।

12. Function Expression-এর ক্ষেত্রে কী হয়?

এটা:

```
greet();

const greet = function () {
  console.log("Hello");
};
```

সাধারণভাবে কাজ করবে না।

কারণ `greet` এখানে `const` variable।

`const` variable initialization-এর আগে access করলে **Temporal Dead Zone (TDZ)**-এর কারণে error হবে।

সুতরাং:

```
const greet = function () {
  console.log("Hello");
};

greet();
```

এটাই correct pattern।

13. Function Declaration vs Expression — Quick Table

Feature	Function Declaration	Function Expression
Syntax	<code>function foo() {}</code>	<code>const foo = function() {}</code>
Named function	Yes	Usually variable name
Hoisting	Function declaration hoisted	Variable rules apply
Before definition call	Generally works	<code>const</code> হলে কাজ করে না
Variable-এর মধ্যে stored	না	হ্যাঁ
Can be anonymous	No	Yes

Feature	Function Declaration	Function Expression
Common use	Traditional named functions	Callbacks / dynamic assignment

14. Anonymous Function

যে function-এর নিজের কোনো নাম নেই তাকে সাধারণভাবে **anonymous function** বলা হয়।

Example:

```
const greet = function () {
  console.log("Hello");
};
```

এখানে function-এর কোনো internal name নেই।

Function expression-এর ক্ষেত্রে anonymous function খুব common।

15. Named Function Expression

Function expression-এর function-এর নিজের নামও থাকতে পারে।

```
const greet = function sayHello() {
  console.log("Hello");
};
```

এখানে:

```
greet
```

হলো variable name।

আর:

```
sayHello
```

হলো function-এর internal name।

Named function expressions debugging/stack traces বা recursive patterns-এ useful হতে পারে।

16. Function Return Value

Function শুধু `console.log()` করবে এমন না।

Function কোনো value **return** করতে পারে।

```
function add(a, b) {  
  return a + b;  
}
```

এখন:

```
const result = add(10, 20);  
  
console.log(result);
```

Output:

30

এখানে function:

```
return a + b;
```

এর মাধ্যমে `30` ফেরত দিয়েছে।

17. `return` কী করে?

`return` দুটি গুরুত্বপূর্ণ কাজ করে:

1. Function থেকে একটি value বাইরে পাঠায়
2. Function execution immediately শেষ করে

Example:

```
function add(a, b) {  
  return a + b;  
  
  console.log("This will not run");  
}
```

`return` execute হওয়ার পর function শেষ।

তাই:

```
console.log("This will not run");
```

execute হবে না।

18. `return` দিয়ে **Value Store** করা

```
function multiply(a, b) {  
  return a * b;  
}  
  
const result = multiply(5, 4);  
  
console.log(result);
```

Output:

```
20
```

Mental model:

```
multiply(5, 4)  
  ↓  
  20  
  ↓  
const result = 20
```

19. return vs console.log()

এটা beginner-দের খুব common confusion ।

`console.log()`

শুধু console-এ value দেখায় ।

```
function add(a, b) {  
  console.log(a + b);  
}  
  
const result = add(10, 20);  
  
console.log(result);
```

Output:

```
30  
undefined
```

কারণ function কিছু return করেনি।

`return`

Value function-এর বাইরে পাঠায়।

```
function add(a, b) {  
  return a + b;  
}  
  
const result = add(10, 20);  
  
console.log(result);
```

Output:

```
30
```

Mental Model

```
console.log()
```

→ দেখাও

```
return
```

→ ফেরত দাও

20. Function Without return

যদি function কোনো value return না করে:

```
function greet() {  
  console.log("Hello");  
}
```

তাহলে:

```
const result = greet();  
  
console.log(result);
```

Output:

```
Hello  
undefined
```

JavaScript implicitly:

```
return undefined;
```

এর মতো behavior করে।

21. return undefined

Explicitly-ও করা যায়:

```
function test() {  
  return undefined;  
}
```

অথবা:

```
function test() {  
  return;  
}
```

দুটোর result:

```
undefined
```

22. `return` করলে **Function Immediately Stop** হয়

```
function checkAge(age) {  
  if (age < 18) {  
    return "Not allowed";  
  }  
  
  return "Allowed";  
}
```

Example:

```
console.log(checkAge(15));
```

Output:

```
Not allowed
```

এখানে প্রথম `return` execute হলে function শেষ।

23. Multiple return

একটি function-এ multiple `return` থাকতে পারে।

```
function getGrade(mark) {  
  if (mark >= 80) {  
    return "A+";  
  }  
  
  if (mark >= 70) {  
    return "A";  
  }  
  
  if (mark >= 60) {  
    return "B";  
  }  
  
  return "Fail";  
}
```

প্রথম matching `return` execute হবে।

24. Guard Clause

`return` ব্যবহার করে function-এর শুরুতেই invalid case handle করা যায়।

এটাকে বলা হয় **Guard Clause**।

```
function processUser(user) {  
  if (!user) {  
    return;  
  }  
  
  if (!user.name) {  
    return;  
  }  
  
  console.log(user.name);  
}
```

এতে nested `if` কমানো যায়।

25. Function Composition-এর Basic Idea

একটি function-এর return value অন্য function-এ পাঠানো যায়।

```
function add(a, b) {  
  return a + b;  
}  
  
function double(number) {  
  return number * 2;  
}  
  
const result = double(add(5, 10));  
  
console.log(result);
```

Flow:

```
add(5, 10)  
  ↓  
  15  
  ↓  
double(15)  
  ↓  
  30
```

এটা পরবর্তীতে functional programming এবং React code-এ অনেক কাজে আসবে।

26. Function Scope — Basic Idea

Function-এর ভিতরে declare করা variable সাধারণত function-এর বাইরে accessible নয়।

```
function test() {  
  const message = "Hello";  
  console.log(message);  
}  
  
test();
```

কিন্তু:

```
console.log(message);
```

এটা কাজ করবে না।

কারণ `message` function-এর local scope-এর মধ্যে আছে।

27. Local Variable

```
function calculate() {  
  const price = 100;  
  const quantity = 2;  
  
  return price * quantity;  
}
```

এখানে:

```
price  
quantity
```

function-এর local variables।

Function শেষ হলে এগুলো বাইরে direct access করা যায় না।

28. Global Variable এবং Function

Function-এর বাইরে declare করা variable function-এর ভিতর থেকে access করা যায়।

```
const tax = 10;  
  
function calculatePrice(price) {  
  return price + tax;  
}  
  
console.log(calculatePrice(100));
```

Output:

110

কারণ function outer scope-এর variable access করতে পারে।

29. Function Parameter-এর Scope

Parameter-ও function-এর local variable-এর মতো কাজ করে।

```
function greet(name) {  
  console.log(name);  
}
```

এখানে:

name

শুধু function-এর scope-এর মধ্যে available।

30. Function-এ Same Name ব্যবহার

Function-এর local variable outer variable shadow করতে পারে।

```
const name = "Global";  
  
function test() {  
  const name = "Local";  
  
  console.log(name);  
}  
  
test();  
  
console.log(name);
```

Output:

Local
Global

Function-এর ভিতরে local `name` ব্যবহার হয়েছে।

এটা **shadowing** নামে পরিচিত।

31. Functions are Values

JavaScript-এ function শুধু "code block" না।

Function একটি **value** হিসেবেও ব্যবহার করা যায়।

Example:

```
function greet() {  
  console.log("Hello");  
}  
  
const myFunction = greet;  
  
myFunction();
```

Output:

Hello

এখানে:

```
const myFunction = greet;
```

function-কে variable-এর মধ্যে রাখা হয়েছে।

32. Function Reference vs Function Call

এটা খুব important।

Reference

```
const fn = greet;
```

এখানে function execute হচ্ছে না।

Call

```
greet();
```

এখানে function execute হচ্ছে।

আর:

```
const fn = greet();
```

এখানে function execute হবে এবং তার **return value** `fn` -এ যাবে।

33. Example — Reference vs Call

```
function greet() {  
  return "Hello";  
}  
  
const a = greet;  
const b = greet();  
  
console.log(a);  
console.log(b);
```

Conceptually:

```
a → function reference  
b → "Hello"
```

কারণ:

```
greet
```

মানে function reference ।

আর:

```
greet()
```

মানে function call ।

34. Function as a Reusable Unit

ধরো ecommerce application:

```
function calculateTotal(price, quantity) {  
  return price * quantity;  
}
```

এখন:

```
calculateTotal(500, 2);  
calculateTotal(1000, 3);  
calculateTotal(250, 5);
```

একই logic বারবার ব্যবহার করা যাচ্ছে ।

35. Pure Function — Basic Concept

Pure function এমন function যার output শুধুমাত্র input-এর ওপর নির্ভর করে এবং বাইরে কোনো observable side effect তৈরি করে না ।

Example:

```
function add(a, b) {  
  return a + b;  
}
```

একই input দিলে একই output:

```
add(2, 3); // 5
add(2, 3); // 5
```

36. Impure Function — Basic Concept

যে function বাইরের state/data পরিবর্তন করে বা external state-এর ওপর নির্ভর করে, সেটাকে সাধারণভাবে impure function বলা হয়।

Example:

```
let total = 0;

function addToTotal(amount) {
  total += amount;
}
```

এখানে function বাইরের:

total

পরিবর্তন করছে।

এটা side effect তৈরি করছে।

37. Pure vs Impure

Pure Function	Impure Function
Same input → same output	Output/state external factor-এর ওপর depend করতে পারে
External state পরিবর্তন করে না	External state পরিবর্তন করতে পারে
Side effect নেই	Side effect থাকতে পারে
Test করা সহজ	Test/behavior track করা তুলনামূলক কঠিন
Functional programming-এ গুরুত্বপূর্ণ	Real-world apps-এ side effects প্রয়োজন হতে পারে

38. Pure Function Example

```
function square(number) {  
  return number * number;  
}
```

এটা pure ।

কারণ:

```
square(5) → 25  
square(5) → 25  
square(5) → 25
```

সবসময় একই result ।

39. Function Design — Single Responsibility

একটি function ideally একটি clear কাজ করবে।

ভালো:

```
function calculateTotal(price, quantity) {  
  return price * quantity;  
}
```

আরেকটি:

```
function formatCurrency(amount) {  
  return `>${amount}`;  
}
```

তারপর combine করা যায়।

```
const total = calculateTotal(500, 2);  
const formatted = formatCurrency(total);
```

এতে code maintain করা সহজ হয়।

40. Function Naming Best Practices

Function-এর নাম সাধারণত এমন হওয়া ভালো যাতে নাম দেখেই কাজ বোঝা যায়।

ভালো:

```
calculateTotal()  
getUser()  
validateEmail()  
formatDate()  
findProduct()  
sendMessage()
```

খারাপ:

```
doIt()  
thing()  
abc()  
test()  
function1()
```

Function-এর নাম সাধারণত **verb/action oriented** হলে ভালো হয়।

41. Function Declaration কখন ব্যবহার করব?

Traditional named reusable logic-এর জন্য function declaration খুব ভালো:

```
function calculateTax(price) {  
  return price * 0.15;  
}
```

এটা:

- readable
 - clear
 - reusable
 - সহজে বোঝা যায়
-

42. Function Expression কখন ব্যবহার করা হয়?

Function-কে variable-এর মধ্যে রাখতে হবে:

```
const calculateTax = function (price) {  
  return price * 0.15;  
};
```

এটা বিশেষ করে useful যখন:

- function dynamically assign করতে হবে
- callback হিসেবে function পাঠানো হবে
- condition অনুযায়ী function পরিবর্তন হবে
- function value হিসেবে ব্যবহার করা হবে

Callback concept আমরা পরে বিস্তারিত দেখব।

43. Conditional Function Assignment

Function expression-এর একটি practical pattern:

```
const isAdmin = true;  
  
const getMessage = isAdmin  
  ? function () {  
    return "Admin";  
  }  
  : function () {  
    return "User";  
  };  
  
console.log(getMessage());
```

এখানে condition অনুযায়ী different function assign করা হচ্ছে।

44. Function Declaration vs Function Expression — Deep Mental Model

Declaration

```
function add(a, b) {  
  return a + b;  
}
```

ভাবো:

"আমি একটা function declare করছি।"

Expression

```
const add = function (a, b) {  
  return a + b;  
};
```

ভাবো:

"আমি একটা function value তৈরি করে variable-এর মধ্যে রাখছি।"

45. Function Call-এর সময় Arguments Matching

```
function add(a, b) {  
  return a + b;  
}  
  
add(10, 20);
```

Mapping:

```
a ← 10  
b ← 20
```

তারপর:

```
10 + 20  
↓  
30
```

46. বেশি **Arguments** দিলে কী হয়?

```
function add(a, b) {  
  return a + b;  
}  
  
console.log(add(10, 20, 30));
```

Function-এর declared parameters:

```
a  
b
```

তাই:

```
a = 10  
b = 20
```

Extra `30` এই function-এর normal parameter list-এ assign হবে না।

পরে **rest parameters** এবং **arguments object** দিয়ে extra arguments handle করা শিখব।

47. কম **Arguments** দিলে কী হয়?

```
function add(a, b) {  
  return a + b;  
}  
  
console.log(add(10));
```

Mapping:

```
a = 10  
b = undefined
```

তাই:

```
10 + undefined  
→ NaN
```

Output:

```
NaN
```

পরে **default parameters** দিয়ে এই problem handle করা শিখব।

48. Function Return করা Object

Function যেকোনো JavaScript value return করতে পারে।

Object:

```
function createUser() {  
  return {  
    name: "Sharlar",  
    age: 31  
  };  
}  
  
const user = createUser();  
  
console.log(user.name);
```

49. Function Return করা Array

```
function getNumbers() {  
  return [10, 20, 30];  
}
```

```
const numbers = getNumbers();  
  
console.log(numbers);
```

50. Function Return করা Function

JavaScript-এ function যেহেতু value, তাই function অন্য function-ও return করতে পারে।

Basic example:

```
function createGreeting() {  
  return function () {  
    console.log("Hello");  
  };  
}  
  
const greet = createGreeting();  
  
greet();
```

এটা পরবর্তীতে **Higher-Order Functions** এবং **Closures** বুঝতে খুব important হবে।

51. Function Return Value Chain

```
function getPrice() {  
  return 100;  
}  
  
function addTax(price) {  
  return price * 1.15;  
}  
  
const finalPrice = addTax(getPrice());  
  
console.log(finalPrice);
```

Flow:

```
getPrice()  
  ↓  
  100  
  ↓  
addTax(100)  
  ↓  
  115
```

52. Common Mistake — `return` ভুল জায়গায়

ভুল:

```
function add(a, b) {  
  console.log(a + b);  
}
```

যদি পরে `result` দরকার হয়:

```
const result = add(10, 20);
```

তাহলে:

```
result = undefined
```

সঠিক:

```
function add(a, b) {  
  return a + b;  
}
```

53. Common Mistake — `return` এর পরে **Code**

```
function test() {  
  return 10;  
}
```

```
console.log("Hello");  
}
```

`return` -এর পরে function আর ওই execution path-এ যাবে না।

তাই `"Hello"` print হবে না।

54. Common Mistake — Function Call ভুলে যাওয়া

```
function greet() {  
  console.log("Hello");  
}  
  
greet;
```

এখানে function execute হবে না।

সঠিক:

```
greet();
```

মনে রাখবে:

```
greet  
→ reference  
  
greet()  
→ call
```

55. Common Mistake — `console.log` কে `return`

ভাবা

ভুল ধারণা:

```
function add(a, b) {  
  console.log(a + b);  
}
```

এটা value return করে না।

সঠিক:

```
function add(a, b) {  
  return a + b;  
}
```

56. Practical Example — Shopping Cart

```
function calculateSubtotal(price, quantity) {  
  return price * quantity;  
}  
  
function calculateDiscount(amount, discountPercent) {  
  return amount - (amount * discountPercent) / 100;  
}  
  
const subtotal = calculateSubtotal(500, 3);  
const finalPrice = calculateDiscount(subtotal, 10);  
  
console.log(finalPrice);
```

Flow:

```
price × quantity  
  ↓  
  subtotal  
  ↓  
  discount  
  ↓  
  final price
```

এভাবে বড় logic ছোট ছোট function-এ ভাগ করা যায়।

57. Practical Example — Login Validation

```
function validateLogin(email, password) {  
  if (!email) {  
    return "Email is required";  
  }  
  
  if (!password) {  
    return "Password is required";  
  }  
  
  return "Login data is valid";  
}
```

এখানে guard clauses ব্যবহার হয়েছে।

58. Practical Example — User Greeting

```
function getGreeting(name) {  
  if (!name) {  
    return "Hello, Guest";  
  }  
  
  return `Hello, ${name}`;  
}  
  
console.log(getGreeting("Shariar"));  
console.log(getGreeting());
```

Output:

```
Hello, Shariar  
Hello, Guest
```

59. Function-এর Basic Anatomy

একটি function:

```
function add(a, b) {  
  return a + b;  
}
```

এভাবে ভাঙতে পারো:

```
function  
  ↓  
keyword  
  
add  
  ↓  
name  
  
(a, b)  
  ↓  
parameters  
  
{  
  return a + b;  
}  
  ↓  
function body
```

Call:

```
add(10, 20);
```

এখানে:

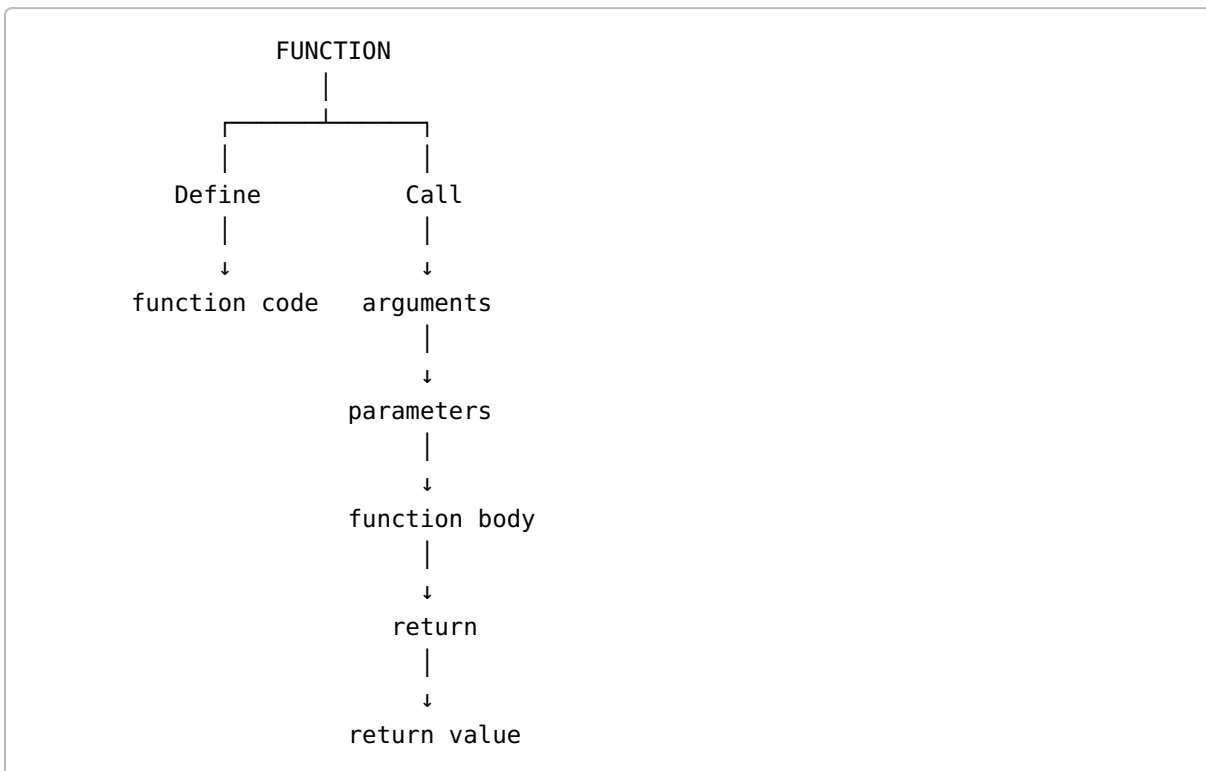
```
10, 20  
  ↓  
arguments
```

Return:

```
30  
  ↓  
return value
```

60. Function-এর Complete Mental Model

সবকিছু একসাথে:



61. Quick Revision

Concept	Meaning
Function	Reusable code block
Function declaration	<code>function add() {}</code>
Function expression	<code>const add = function() {}</code>
Function call	Function execute করা
Parameter	Definition-এর variable
Argument	Call-এর actual value
<code>return</code>	Function থেকে value ফেরত দেয়
<code>console.log()</code>	Console-এ value দেখায়
No return	<code>undefined</code>

Concept	Meaning
<code>break</code>	Loop শেষ করে
<code>return</code>	Function শেষ করে
Local variable	Function-এর ভিতরের variable
Pure function	Same input → same output, no relevant side effect
Function reference	Function-কে execute না করে refer করা

62. Interview Questions

Q1. Function কী?

Function হলো reusable block of code যা নির্দিষ্ট কাজ সম্পন্ন করে এবং প্রয়োজন অনুযায়ী invoke করা যায়।

Q2. Function declaration কী?

```
function add(a, b) {
  return a + b;
}
```

এটি function declaration।

Q3. Function expression কী?

```
const add = function (a, b) {
  return a + b;
};
```

এখানে function-কে একটি expression হিসেবে variable-এ assign করা হয়েছে।

Q4. Parameter আর argument-এর difference কী?

```
function add(a, b) {}
```

`a`, `b` হলো parameters।

```
add(10, 20);
```

10, 20 হলো arguments।

Q5. `return` এবং `console.log()`-এর difference কী?

`console.log()` value console-এ দেখায়।

`return` value function-এর বাইরে পাঠায় এবং function execution শেষ করে।

Q6. Function declaration আগে **call** করা যায় কেন?

Function declarations execution-এর আগে hoisted/available থাকে।

Q7. Function expression আগে **call** করলে কী হয়?

যদি `const` / `let` দিয়ে assign করা হয়, initialization-এর আগে access করলে TDZ-related `ReferenceError` হয়।

Q8. Function কিছু **return** না করলে কী পাওয়া যায়?

```
undefined
```

Q9. `return` **execute** হলে কী হয়?

Function immediately শেষ হয়ে যায়।

Q10. JavaScript-এ function কি **value**?

হ্যাঁ। Function variable-এ store করা, argument হিসেবে পাঠানো এবং return করা যায়।

63. Practice — Output Predict করো

Question 1

```
function greet() {  
  console.log("Hello");  
}  
  
greet();
```

Output কী?

Question 2

```
function add(a, b) {  
  return a + b;  
}  
  
const result = add(5, 10);  
  
console.log(result);
```

Output কী?

Question 3

```
function test() {  
  console.log("A");  
  return 10;  
  console.log("B");  
}  
  
console.log(test());
```

Output কী হবে?

Question 4

```
function add(a, b) {  
  console.log(a + b);  
}  
  
const result = add(10, 20);  
  
console.log(result);
```

কেন `undefined` আসবে?

Question 5

```
function add(a, b) {  
  return a + b;  
}  
  
console.log(add(10));
```

Output কী হবে এবং কেন?

Question 6

```
function greet(name) {  
  return `Hello ${name}`;  
}  
  
const fn = greet;  
  
console.log(fn("Shariar"));
```

এখানে `fn` কী?

64. Coding Practice

Challenge 1 — Square

একটি function তৈরি করো:

```
square(5);
```

যার output হবে:

```
25
```

Challenge 2 — Maximum

দুইটি number-এর মধ্যে বড় number return করবে:

```
max(10, 20);
```

Expected:

```
20
```

Challenge 3 — Even Check

```
isEven(10);
```

Expected:

```
true
```

আর:

```
isEven(7);
```

Expected:

```
false
```

Challenge 4 — Calculate Total

```
calculateTotal(500, 3);
```

Expected:

```
1500
```

Challenge 5 — Grade

একটি function:

```
getGrade(85);
```

Expected:

```
A+
```

তোমার condition অনুযায়ী অন্যান্য grade-ও handle করবে।

65. Golden Rules

Rule 1

Function define করা আর call করা এক জিনিস না।

```
define → তৈরি  
call → execute
```

Rule 2

Parameter হলো function-এর input variable।

Argument হলো actual value।

```
function add(a, b) {}  
add(10, 20);
```

Rule 3

`return` value ফেরত দেয়।

```
return result;
```

Rule 4

`console.log()` return করে না।

```
console.log(value);
```

শুধু display করে।

Rule 5

`return` function execution শেষ করে।

Rule 6

Function declaration এবং function expression-এর hoisting behavior এক নয়।

Rule 7

Function JavaScript-এ first-class value।

অর্থাৎ function:

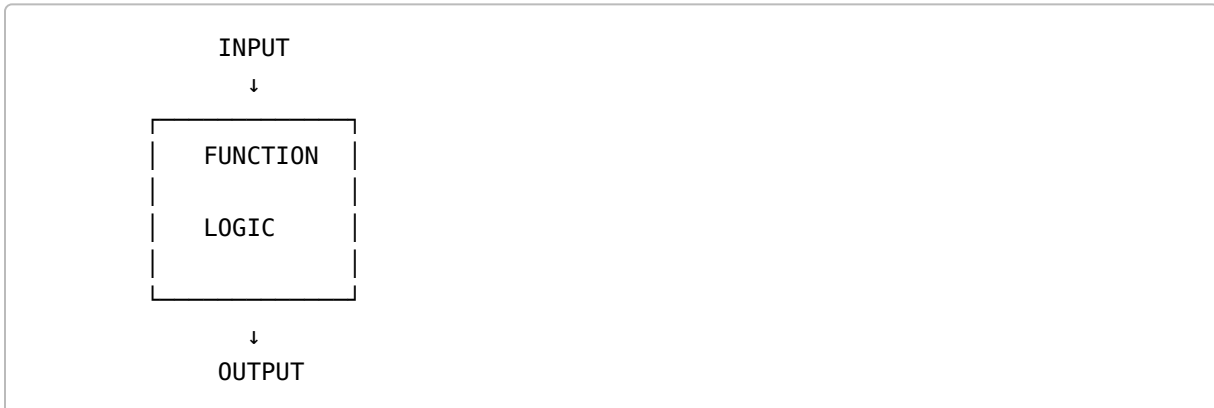
```
store করা যায়  
pass করা যায়  
return করা যায়
```

Rule 8

একটি function-এর ideally একটি clear responsibility থাকা ভালো।

66. Final Mental Model

একটা function-কে তুমি একটা **machine** হিসেবে ভাবতে পারো:



Example:

```
function multiply(price, quantity) {  
  return price * quantity;  
}
```

```
price = 500  
quantity = 3  
  ↓  
  FUNCTION  
  ↓  
500 × 3  
  ↓  
1500
```

এই mental modelটা মাথায় রাখলে পরের concepts অনেক সহজ হবে।

🚩 Doc 10.1 Complete

এই doc-এর core concepts:

```
Function
↓
Declaration
↓
Expression
↓
Call / Invocation
↓
Parameters
↓
Arguments
↓
Return
↓
Scope
↓
Function as a value
↓
Pure / Impure basics
```

পরের doc-এ আমরা যাব:

Doc 10.2 — Arrow Functions & Default Parameters

যেখানে থাকবে:

```
Arrow Function
↓
Basic Syntax
↓
Parameters
↓
Implicit Return
↓
Explicit Return
↓
Object Return
↓
Arrow vs Regular Function
↓
Default Parameters
↓
undefined behavior
```

↓

Default vs || vs ??

↓

React practical patterns